

Exact Structural Degrees of Freedom for Multiway High-Dimensional Categorical Fixed Effects

A rank-theoretic and hypergraph reduction approach for arbitrary numbers of fixed-effect partitions

Anonymous technical manuscript

Implementation reference: `econhdfc/hdfc`, `exactdof2` prototype

September 2026

Abstract

High-dimensional fixed-effect (HDFE) estimators can absorb an arbitrary number of categorical effects, but exact accounting of the degrees of freedom consumed by three or more fixed-effect partitions is not generally available in widely used HDFE software. Current `reghdfc` documentation, for example, describes exact graph-theoretic accounting for the first two fixed effects and a conservative pairwise approximation thereafter. This note develops a general exact formulation for intercept-only categorical HDFE designs. The absorbed structural degrees of freedom are the rank, over characteristic zero, of the concatenated dummy matrix. After duplicate observation patterns are removed, this matrix is the edge–vertex incidence matrix of a multipartite uniform hypergraph. We establish a sequence of rank-preserving reductions: component decomposition, exact degree-one peeling, removal of the universal block-sum dependencies, and a sufficient “proper connectivity” certificate that yields rank in closed form. Residual cores are certified by finite-field rank whenever the finite-field lower bound reaches a deterministic characteristic-zero upper bound; otherwise an exact integer/rational backend is invoked. The resulting procedure is exact for any finite number of categorical fixed-effect partitions, contains the standard two-way connected-component formula as a special case, and separates mathematical identification from numerical projection of fixed effects. We also give a minimal three-way counterexample showing why pairwise connected-component information is insufficient in general. The note states the assumptions, proofs, computational complexity, implementation correspondence, and scope limitations needed for a reproducible software implementation.

Formal verification and priority boundary (September 2026). The core mathematical statements in this note are mapped to an independent Lean 4 proof library and have passed complete compilation and per-theorem axiom audit under the assumptions recorded in `formal/mathematical-coverage.md`. This machine-checked status concerns the mathematics, not the correctness of the Python/Numba implementation, floating-point behavior, resource management, external-package parity, or historical priority. Established linear algebra and graph/hypergraph ingredients retain their prior-art status; Lean verification does not establish novelty.

Keywords: high-dimensional fixed effects; degrees of freedom; categorical effects; hypergraph incidence matrix; matrix rank; exact linear algebra; finite fields.

JEL codes: C01, C13, C63, C87.

Claim boundary. The results below concern the structural rank of *intercept-only categorical* fixed-effect blocks. Heterogeneous slopes, continuous interactions, cluster-nesting conventions used

in small-sample variance corrections, and convergence of the numerical HDFE projection solver are separate issues unless explicitly stated otherwise.

Terminology and input contract (0.6.1 review). Here “structural rank” means the actual characteristic-zero rank of the specified zero–one categorical design, not the generic rank of its sparsity pattern. For the complete 2×2 two-way design the latter is four but the actual rank is three. Observed levels only are counted; a declared dense coding must be a finite nonnegative integer coding with no unobserved holes. A full-rank proper-connectivity certificate is sufficient, not necessary. These distinctions are covered by exact-arithmetic regression tests in the 0.6.1 review.

1 Introduction

Large administrative and panel datasets routinely motivate linear and nonlinear models with several high-dimensional categorical fixed effects. Computational methods for partialling out these effects are by now well developed; prominent contributions include Guimarães and Portugal (2010), Gaure (2013), and Correia (2017). For two fixed-effect partitions, graph connectivity gives an exact characterization of the number of linearly independent fixed-effect coefficients; this insight is central in matched worker–firm models such as Abowd et al. (2002). Estimation with three or more fixed effects is also computationally feasible, but degrees-of-freedom accounting is more delicate. Guimarães’s `group3hdf` provides a dedicated exact procedure for three HDFEs (Guimarães, 2016), whereas current `reghdfe` documentation continues to describe the general problem beyond two fixed effects as open and uses pairwise connected-component calculations as a conservative approximation (Correia and Constantine, 2026).

This note takes a deliberately algebraic view. The object of interest is not an informal count of dummy coefficients but the dimension of the fixed-effect column space. For categorical intercept effects, that dimension is exactly the rank of the concatenated dummy matrix. Once repeated fixed-effect tuples are removed, the same matrix can be interpreted as the edge–vertex incidence matrix of a multipartite, uniform hypergraph. This representation makes it possible to combine graph reductions and exact linear algebra without changing the estimand.

The contribution is therefore primarily a general computational framework rather than a claim that the rank identity itself is novel. The framework has four features. First, it applies to an arbitrary finite number of categorical fixed-effect partitions. Second, every structural simplification is accompanied by a rank identity, so preprocessing cannot change the answer. Third, finite-field computation is used only as a *certificate*: a modular rank is accepted only when it attains a deterministic upper bound on the characteristic-zero rank. Fourth, difficult residual cores fall back to exact integer/rational linear algebra, avoiding floating-point rank tolerances.

The theory below also clarifies why a purely pairwise method cannot be exact in general. With three or more partitions, the design is a hypergraph incidence problem. Two designs can have the same pairwise connectivity information but different collective linear dependencies. Exact accounting must therefore inspect the joint incidence structure, not only its pairwise projections.

2 Model and target quantity

Consider the linear model

$$y = X\beta + \sum_{g=1}^G D_g \alpha_g + \varepsilon, \quad (1)$$

where $D_g \in \{0, 1\}^{N \times K_g}$ is the one-hot design matrix for the g th categorical fixed-effect partition. Every observation belongs to exactly one observed level in each partition. Let

$$D = [D_1 \ D_2 \ \cdots \ D_G], \quad K = \sum_{g=1}^G K_g. \quad (2)$$

A separate constant, if supplied, lies in the column span of every nonempty complete dummy block and therefore does not enlarge the fixed-effect span.

Assumption 2.1 (Observed-level coding). *Each D_g contains columns only for levels observed in the estimation sample. If integer level codes are passed to an implementation under a “dense-code” contract, they take values in $\{0, \dots, K_g - 1\}$ and every code in that set occurs at least once; repeated codes across observations are permitted.*

Definition 2.2 (Structural absorbed degrees of freedom). The structural degrees of freedom absorbed by the categorical fixed effects are

$$d_a^{\text{struct}} := \dim \text{col}(D) = \text{rank}_{\mathbb{R}}(D). \quad (3)$$

This definition is the natural parameter-space dimension consumed by the fixed-effect design. Regression packages can apply additional inference conventions—for example, treating a fixed effect nested in a cluster variable as costless for a particular small-sample cluster correction. Such conventions alter an inference adjustment, not the structural rank derived here.

Lemma 2.3 (Characteristic-zero equivalence). *For the integer matrix D ,*

$$\text{rank}_{\mathbb{R}}(D) = \text{rank}_{\mathbb{Q}}(D). \quad (4)$$

Proof. The rank of a matrix over a field equals the largest order of a nonzero minor. Every minor of D is an integer. An integer determinant is zero in $\mathbb{R}$ if and only if it is zero in $\mathbb{Q}$. Hence the largest nonzero minor has the same order over both fields. $\square$

Thus exact DoF computation is an exact rational/integer rank problem; no numerical rank tolerance is required.

2.1 Prefix ranks and blockwise redundancy

Some software reports the contribution of each fixed-effect block sequentially. Define

$$R_j = \text{rank}_{\mathbb{Q}}([D_1, \dots, D_j]), \quad R_0 = 0. \quad (5)$$

The incremental dimension of block j is $\Delta_j = R_j - R_{j-1}$. Its exact redundancy count, under this ordering, is

$$M_j = K_j - \Delta_j. \quad (6)$$

Then

$$\sum_{j=1}^G (K_j - M_j) = \sum_{j=1}^G \Delta_j = R_G = d_a^{\text{struct}}. \quad (7)$$

The allocation $(M_1, \dots, M_G)$ depends on block order, but the total R_G does not.

3 Hypergraph representation

For observation i , write its fixed-effect tuple as

$$e_i = (v_{i1}, \dots, v_{iG}), \quad v_{ig} \in V_g, \quad (8)$$

where V_g is the set of observed levels in partition g . Let $\mathcal{V} = \dot{\bigcup}_{g=1}^G V_g$ be the disjoint union of all fixed-effect levels. Every observation defines a hyperedge containing exactly one vertex from each V_g . Hence the fixed-effect design is naturally associated with a G -partite, G -uniform hypergraph

$$\mathcal{H} = (\mathcal{V}, \mathcal{E}). \quad (9)$$

After duplicate tuples are removed, the resulting edge–vertex incidence matrix $A \in \{0, 1\}^{|\mathcal{E}| \times |\mathcal{V}|}$ is exactly the matrix of unique rows of D .

Lemma 3.1 (Duplicate-edge reduction). *Deleting repeated fixed-effect tuples does not change rank. If A contains one copy of each distinct row of D , then*

$$\text{rank}(D) = \text{rank}(A). \quad (10)$$

Proof. Repeated rows do not enlarge the row space. The row spaces of D and A are therefore identical. $\square$

This reduction can be substantial in large panels because N may be much larger than the number $E = |\mathcal{E}|$ of unique fixed-effect tuples.

4 Exact structural reductions

4.1 Connected-component decomposition

Use ordinary incidence connectivity: two vertices are in the same component if they can be linked by a sequence of hyperedges with nonempty intersections. Because every edge contains one vertex from each partition, every nonempty incidence component contains at least one vertex from every partition.

Lemma 4.1 (Component additivity). *Suppose $\mathcal{H}$ has incidence-connected components $\mathcal{H}_1, \dots, \mathcal{H}_C$, with corresponding incidence submatrices $A_1, \dots, A_C$. Then*

$$\text{rank}(A) = \sum_{c=1}^C \text{rank}(A_c). \quad (11)$$

Proof. Permute rows by edge component and columns by vertex component. No edge in one component is incident to a vertex in another, so the permuted matrix is block diagonal. The rank of a block-diagonal matrix is the sum of the block ranks. $\square$

4.2 Degree-one peeling

A vertex has active degree one if it occurs in exactly one active unique hyperedge.

Lemma 4.2 (Exact leaf elimination). *Let A be an incidence matrix and suppose column v contains exactly one nonzero entry, located in row e . Let A' be obtained by deleting row e and column v . Then*

$$\text{rank}(A) = 1 + \text{rank}(A'). \quad (12)$$

Consequently, recursively peeling a degree-one vertex together with its unique incident edge contributes exactly one unit of rank per peeled edge.

Proof. After permuting row e and column v to the first positions,

$$A = \begin{bmatrix} 1 & r' \\ 0 & A' \end{bmatrix}. \quad (13)$$

The first row is linearly independent of all remaining rows because it is the only row with a nonzero entry in the first column. The remaining row rank equals $\text{rank}(A')$. $\square$

Remark 4.3. The relevant degree is degree in the *unique-edge* hypergraph, not the raw observation count. A fixed-effect level can appear in thousands of observations but still be a degree-one vertex after all those observations collapse to one repeated fixed-effect tuple.

Repeated application of Lemma 4.2 produces a residual 2-core-like hypergraph in which each active vertex has degree at least two. If P edges are peeled and A_{core} denotes the residual incidence matrix, then

$$\text{rank}(A) = P + \text{rank}(A_{\text{core}}). \quad (14)$$

4.3 Universal block-sum dependencies

Consider one nonempty incidence-connected component with E_c edges and V_c vertices. Let V_{cg} be its vertices in partition g , and let $s_g \in \{0, 1\}^{V_c}$ indicate the columns belonging to partition g . Because every row has exactly one entry equal to one in every partition,

$$A_c s_g = \mathbf{1}_{E_c}, \quad g = 1, \dots, G. \quad (15)$$

Therefore

$$A_c(s_g - s_1) = 0, \quad g = 2, \dots, G. \quad (16)$$

The $G - 1$ vectors $s_g - s_1$ are linearly independent because the partition supports are disjoint.

Proposition 4.4 (Deterministic multipartite upper bound). *For every nonempty connected G -partite, G -uniform incidence component,*

$$\text{rank}(A_c) \leq U_c := \min\{E_c, V_c - G + 1\}. \quad (17)$$

Proof. The row count gives $\text{rank}(A_c) \leq E_c$. The $G - 1$ independent null vectors above imply column nullity at least $G - 1$, hence $\text{rank}(A_c) \leq V_c - (G - 1) = V_c - G + 1$. $\square$

Lemma 4.5 (Exact column reduction). *Within a nonempty component, choose one arbitrary column $b_g \in V_{cg}$ from each partition $g = 2, \dots, G$. Deleting these $G - 1$ columns does not change the column span or rank.*

Proof. For every $g \geq 2$,

$$d_{g,b_g} = \sum_{v \in V_{c1}} d_{1,v} - \sum_{v \in V_{cg} \setminus \{b_g\}} d_{g,v}, \quad (18)$$

where $d_{g,v}$ denotes the incidence column of vertex v . Thus every deleted column is an exact linear combination of retained columns. $\square$

This reduction leaves exactly $V_c - G + 1$ columns, which is the non-row-count part of the upper bound in (17).

5 A structural full-rank certificate

The implementation contains a particularly cheap structural certificate. Define two hyperedges to be *one-coordinate adjacent* if they share exactly $G - 1$ vertices, equivalently if their fixed-effect tuples differ in one coordinate only. Let $L(\mathcal{H}_c)$ be the graph whose nodes are hyperedges and whose edges join one-coordinate-adjacent hyperedges.

Definition 5.1 (Proper edge connectivity). A residual component is properly edge-connected if $L(\mathcal{H}_c)$ is connected.

Proposition 5.2 (Proper-connectivity rank formula). *If a nonempty G -partite, G -uniform component is properly edge-connected, then*

$$\text{rank}_{\mathbb{Q}}(A_c) = V_c - G + 1. \quad (19)$$

Proof. Fix a root hyperedge $e^0 = (v_1^0, \dots, v_G^0)$. For any vertex $v \in V_{cg}$, choose an edge e containing v . Proper edge connectivity provides a path

$$e^0 = e^{(0)}, e^{(1)}, \dots, e^{(m)} = e \quad (20)$$

where consecutive edges differ in exactly one coordinate. If step t changes partition h , then the difference between the two incidence rows is

$$a_{e^{(t)}} - a_{e^{(t-1)}} = \mathbf{e}_{v_h^{(t)}} - \mathbf{e}_{v_h^{(t-1)}}, \quad (21)$$

which is a within-partition contrast and belongs to the row space of A_c . Summing only the path steps that change coordinate g telescopes to

$$\mathbf{e}_v - \mathbf{e}_{v_g^0}. \quad (22)$$

Hence the row space contains, for every partition g , all $|V_{cg}| - 1$ contrasts relative to the root vertex. Across partitions these give $V_c - G$ linearly independent vectors. The root row a_{e^0} is independent of their span because every within-partition contrast has coordinate sum zero within each partition, whereas a_{e^0} has coordinate sum one in each partition. Thus

$$\text{rank}(A_c) \geq (V_c - G) + 1 = V_c - G + 1. \quad (23)$$

Proposition 4.4 gives the reverse inequality. $\square$

Remark 5.3. This proposition is proved directly for the condition implemented in the software. It does not require importing a stronger or differently defined hypergraph “proper connectivity” theorem. Related incidence-rank phenomena are studied more broadly in the hypergraph literature; see, for example, Parui (2025).

6 Finite-field certificates and exact fallback

Structural reductions do not always determine the rank of a residual core. The remaining task is exact rank computation. The key computational observation is that modular rank supplies a rigorous lower bound on rational rank.

Lemma 6.1 (Finite-field lower bound). *Let B be an integer matrix and p a prime. Then*

$$\text{rank}_{\mathbb{F}_p}(B \bmod p) \leq \text{rank}_{\mathbb{Q}}(B). \quad (24)$$

Proof. If $\text{rank}_{\mathbb{F}_p}(B) = r$, then some $r \times r$ minor has determinant nonzero modulo p . The same determinant, viewed as an integer, is therefore nonzero. Hence the corresponding minor is nonzero over $\mathbb{Q}$, implying $\text{rank}_{\mathbb{Q}}(B) \geq r$. $\square$

Theorem 6.2 (Exact modular certificate). *Let U be any deterministic upper bound on $\text{rank}_{\mathbb{Q}}(B)$. If, for some prime p ,*

$$\text{rank}_{\mathbb{F}_p}(B \bmod p) = U, \quad (25)$$

then

$$\text{rank}_{\mathbb{Q}}(B) = U. \quad (26)$$

Proof. By Lemma 6.1,

$$U = \text{rank}_{\mathbb{F}_p}(B \bmod p) \leq \text{rank}_{\mathbb{Q}}(B) \leq U. \quad (27)$$

All inequalities must therefore hold with equality. $\square$

The implementation first uses $p = 2$ because rows are sparse binary incidence vectors and can be represented efficiently as machine-independent bitsets using integer XOR. Crucially, a shortfall over $\mathbb{F}_2$ is *not* interpreted as the answer. Rank can fall after reduction modulo a prime. If the $\mathbb{F}_2$ pass fails to reach the upper bound, the algorithm proceeds to another exact backend.

A second large-prime modular pass can play the same certification role. If it also fails to attain the upper bound, the implementation computes characteristic-zero rank exactly, either through an external exact algebra package or through a native rational/integer elimination routine. SymPy’s `DomainMatrix` supports exact sparse domains (SymPy Development Team, 2026); FLINT provides compiled exact integer matrix rank, although its `fmpz_mat` representation is dense and therefore requires a memory guard (FLINT Development Team, 2026).

Remark 6.3 (No Monte Carlo acceptance rule). The algorithm never accepts a modular rank merely because the prime is “large.” A modular result is final only when it meets a deterministic characteristic-zero upper bound. Otherwise the fallback computes rank over $\mathbb{Q}$ (or an exact integer domain with the same characteristic-zero rank). Therefore unlucky primes affect speed, not correctness.

7 Main exactness theorem

We can now state the complete result implemented by the categorical rank engine.

Theorem 7.1 (Exact arbitrary- G categorical HDFE rank). *Under Assumption 2.1, consider any finite intercept-only categorical fixed-effect design with $G \geq 1$ partitions. Apply the following operations:*

- (i) *remove duplicate fixed-effect tuples;*

- (ii) recursively peel degree-one vertices and their unique incident edges, adding one to the rank count per peeled edge;
- (iii) decompose the residual incidence hypergraph into connected components;
- (iv) within each residual component, use either the proper-connectivity formula of Proposition 5.2, a finite-field certificate that attains the upper bound in Proposition 4.4, or an exact characteristic-zero rank backend.

If every fallback rank computation terminates, the returned value equals

$$d_a^{\text{struct}} = \text{rank}_{\mathbb{R}}(D). \quad (28)$$

Proof. Lemma 3.1 preserves the initial rank. Every peeling step satisfies the exact recursion in Lemma 4.2; after P steps the original rank is P plus residual-core rank. Lemma 4.1 makes the residual-core rank additive across components. For each component, Proposition 5.2 is exact when its structural condition holds. Otherwise, Theorem 6.2 is exact whenever a modular rank reaches the deterministic upper bound, and by construction the remaining fallback computes the characteristic-zero rank exactly. Summing all exact contributions yields $\text{rank}_{\mathbb{Q}}(D)$, which equals $\text{rank}_{\mathbb{R}}(D)$ by Lemma 2.3. $\square$

8 Relationship to the standard two-way result

For $G = 2$, every connected component is an ordinary bipartite graph. A connected graph with at least one edge has a connected edge-adjacency (line) graph, so Proposition 5.2 gives $\text{rank } V_c - 1$ for each component. Summing over C components yields the familiar formula.

Corollary 8.1 (Two-way connected-component formula). *Let $D = [D_1, D_2]$ contain $K_1 + K_2$ observed dummy columns and let the associated bipartite graph have C connected components. Then*

$$\text{rank}(D) = K_1 + K_2 - C. \quad (29)$$

This is the graph-theoretic result used in two-way matched-effects applications such as Abowd et al. (2002) and in HDFE software. Thus the general rank engine does not replace the standard two-way theory with a different definition; it contains that theory as a special case.

9 Why pairwise connectivity is insufficient for three or more fixed effects

The failure of pairwise methods is easiest to see in a small example. Consider three partitions A , B , and C with levels

$$A = \{A_0, A_1\}, \quad B = \{B_0, B_1\}, \quad C = \{C_0, C_1, C_2\},$$

and five unique tuples

$$\begin{aligned} &(A_0, B_0, C_0), \quad (A_0, B_0, C_1), \quad (A_0, B_1, C_2), \\ &(A_1, B_1, C_0), \quad (A_1, B_1, C_1). \end{aligned}$$

The associated incidence matrix, with columns ordered $(A_0, A_1, B_0, B_1, C_0, C_1, C_2)$, is

$$A = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}. \quad (30)$$

All three pairwise projections A – B , A – C , and B – C are connected. A pairwise mobility-group rule therefore identifies only the universal normalization-type redundancies and yields an absorbed-DoF count of five in the common sequential accounting. Direct exact rank gives

$$\text{rank}_{\mathbb{Q}}(A) = 4. \quad (31)$$

There is one additional collective three-way dependency invisible to pairwise connectivity. For example, the row vector

$$(1, -1, 0, -1, 1)' \quad (32)$$

annihilates A on the left. Equivalently, the full design has three independent column-null relations rather than only the two universal block-sum relations.

This example is not a pathology of floating-point arithmetic. It is an exact integer-rank difference. Therefore no algorithm that observes only pairwise connected-component counts can, in general, recover the full multiway rank.

10 Algorithmic specification

For clarity, the exact categorical rank routine can be summarized as follows.

Algorithm 1: Certified exact categorical HDFE rank

1. Dense-factorize observed levels within each FE partition.
2. Replace repeated FE tuples by one unique hyperedge.
3. Repeatedly peel any degree-one vertex and its unique incident edge; accumulate one rank unit per peeled edge.
4. If no residual edge remains, return the accumulated rank.
5. Split the residual hypergraph into incidence-connected components.
6. For each component, set $U = \min(E_c, V_c - G + 1)$.
7. If one-coordinate edge adjacency is connected, add $V_c - G + 1$.
8. Otherwise remove $G - 1$ universal block-sum columns, preserving rank.
9. Attempt a GF(2) bitset rank. If it reaches U , add U .
10. Otherwise invoke an exact fallback: bounded dense FLINT, sparse SymPy `DomainMatrix`, or native exact modular/rational elimination. Add its exact rank.
11. Return peeled rank plus the sum of residual component ranks.

The order among exact fallback backends is an engineering choice and does not affect the mathematical result.

11 Computational complexity and scaling

Let N be the number of estimation observations, E the number of unique fixed-effect tuples, $V = \sum_g K_g$ the number of observed levels, and G the number of fixed-effect partitions.

Tuple deduplication. A sort/unique implementation requires approximately $O(N \log N)$ comparison work in the generic case, or lower practical cost when tuples can be packed into integer keys. Hash-based alternatives can achieve expected linear time. The crucial output size is E , not N .

Peeling. With linked incidence lists, degree-one peeling is $O(GE + V)$ because every edge-vertex incidence is processed a constant number of times.

Component decomposition. Union-find over vertices touched by each edge requires $O(GE \alpha(V))$ amortized time, where α is the inverse Ackermann function.

Proper-connectivity test. A straightforward implementation forms signatures obtained by omitting one coordinate at a time and groups equal signatures. With sorting/unique operations, this is roughly $O(GE \log E)$ in the worst case, though vectorized implementations have favorable constants.

GF(2) certificate. Bitset elimination is sparse in rows but can densify algebraically. Practical implementations therefore impose work budgets. Because a failure to reach U is never accepted as final, early termination under a work budget affects performance only, not correctness.

Exact fallback. Worst-case exact elimination can be cubic in the smaller matrix dimension and can suffer fill-in and integer growth. This is the remaining hard regime: a very large irreducible core whose characteristic-zero rank lies below the structural upper bound and for which a cheap finite-field pass does not certify the answer. Compiled sparse black-box methods such as Wiedemann-type algorithms are natural future backends for this case, but are not required for correctness of Algorithm 1.

12 Implementation correspondence

Table 1 maps the mathematical claims to the audited `econhdfc/hdfc/rank.py` prototype. This table is included to make the software theorem auditable: every rank-changing code path should correspond to an explicit result above.

12.1 Backend safety

Two implementation details are mathematically important.

First, the GF(2) routine may stop early when a preset XOR budget is exhausted. The partial rank then remains only a lower bound and is discarded unless it has already reached U . Thus the budget cannot generate a false rank.

Second, the FLINT backend is guarded by a maximum dense-cell count because `fmpz_mat` is a dense integer matrix representation (FLINT Development Team, 2026). Refusing a too-large dense conversion is a resource decision, not a change in the algebraic answer; another exact backend is then used.

Table 1: Mathematical result and implementation correspondence

Mathematical operation	Implementation routine	Justification
Remove duplicate tuples	<code>_unique_edge_indices</code>	Lemma 3.1
Create disjoint vertex ids	<code>_global_edges</code>	Representation only
Degree-one elimination	<code>_peel_degree_one</code>	Lemma 4.2
Residual component split	<code>_component_edge_lists</code>	Lemma 4.1
One-coordinate connectivity	<code>_properly_connected</code>	Proposition 5.2
Drop $G - 1$ block columns	<code>_reduced_component_rows</code>	Lemma 4.5
GF(2) bitset certificate	<code>_gf2_bitset_rank</code>	Theorem 6.2
Large-prime sparse certificate	<code>_modular_sparse_rank</code>	Theorem 6.2
Sparse exact fallback	<code>_sympy_sparse_rank</code>	Exact characteristic-zero rank
Dense compiled fallback	<code>_flint_dense_rank</code>	Exact integer rank
Dependency-free fallback	<code>_rational_sparse_rank</code>	Exact row operations over $\mathbb{Q}$
Top-level orchestration	<code>categorical_rank</code>	Theorem 7.1
Prefix allocation	<code>categorical_prefix_ranks</code>	Equation (6)

13 Validation strategy

The mathematical transformations used by the rank framework are separately machine-checked in Lean, but implementation errors remain possible because program correctness is not part of that formalization. The prototype was therefore checked against direct exact rank on small designs. The validation suite associated with the audited module revision includes:

- (a) the three-FE counterexample in Section 8, whose exact rank is four;
- (b) exhaustive enumeration of all 255 nonempty subsets of the $2 \times 2 \times 2$ three-way hyperedge universe, compared with exact rational rank;
- (c) random small three- and four-FE designs compared with exact rational elimination;
- (d) a deliberately chosen matrix for which $\text{rank}_{\mathbb{F}_2} < \text{rank}_{\mathbb{Q}}$, verifying that GF(2) shortfall triggers fallback rather than an incorrect answer;
- (e) a one-million-observation leaf-rich design verifying that duplicate reduction and peeling operate on topology rather than raw sample size;
- (f) backend dispatch checks for the optional exact libraries.

These tests are implementation evidence, not substitutes for the theorems above.

14 Interaction with HDFE estimation and inference

The exact rank engine is conceptually separate from the numerical operation that residualizes y and X with respect to D . Alternating projections, conjugate-gradient acceleration, LSMR, or other HDFE solvers can all compute the same projection onto $\text{col}(D)$ while using different numerical algorithms (Guimarães and Portugal, 2010; Gaure, 2013; Correia, 2017). The present results concern only the dimension of that column space.

Similarly, positive observation weights do not alter structural rank: if W is diagonal with strictly positive entries, $W^{1/2}$ is invertible and

$$\text{rank}(W^{1/2}D) = \text{rank}(D). \quad (33)$$

Zero-weight observations should be removed before constructing the estimation-sample design.

Cluster-robust small-sample corrections require additional conventions. In particular, `reghdfe` treats fixed effects nested within clustering variables specially when computing reported absorbed DoF (Correia and Constantine, 2026). Such nesting adjustments should be applied *after* or around the structural rank engine according to the desired inference convention; they are not statements about the rank of D itself.

15 Scope limitations

The main theorem deliberately does not cover the following cases without additional analysis.

- (i) **Heterogeneous slopes.** Columns such as $1\{g = k\}x$ are not binary incidence columns. Their redundancies can depend on realized continuous values and require separate structural or exact numerical rank analysis.
- (ii) **Multi-membership/group-individual designs.** If an observation can contain multiple members from the same conceptual fixed-effect block, the matrix remains sparse and exact-rank methods still apply, but it is no longer a G -partite G -uniform incidence matrix. The upper bound and proper-connectivity certificate must be modified.
- (iii) **Solver convergence.** Exact DoF does not prove convergence or accuracy of the HDFE projection algorithm.
- (iv) **Cluster and finite-sample inference conventions.** These are software/statistical choices layered on top of structural rank.
- (v) **Extremely large deficient residual cores.** The result remains exact, but runtime can be dominated by the characteristic-zero fallback. A sparse compiled black-box rank backend would improve this regime without changing the theory.

16 Discussion

The central simplification is conceptual: multiway categorical HDFE degrees of freedom are a matrix-rank problem. Two-way graph connectivity solves a special case because the bipartite incidence matrix has a simple rank formula. For three or more partitions, pairwise graphs discard joint incidence information and can miss collective dependencies. Treating the design as a multipartite hypergraph preserves the complete structure while enabling aggressive exact reductions before any general-purpose linear algebra is attempted.

The resulting design principle is conservative in the numerical sense but not in the statistical count: use topology to prove as much rank as possible, use finite fields only for one-sided certificates, and invoke characteristic-zero exact arithmetic whenever certification fails. There is no floating-point tolerance whose choice can change the reported structural DoF.

From a software-engineering perspective, this also argues for keeping the rank engine modular. The same categorical rank kernel can be used by OLS, IV, PPML, or other estimators that absorb

the same categorical column space. Solver optimizations such as canonicalization may simplify the numerical plan, but inference should retain a clearly defined requested-design representation so that purely computational rewrites do not silently change degrees-of-freedom conventions.

17 Conclusion

For intercept-only categorical fixed effects, exact absorbed structural degrees of freedom with any finite number of partitions are computable as the characteristic-zero rank of the concatenated dummy design. The hypergraph representation yields a practical exact algorithm: deduplicate tuples, peel degree-one incidences, decompose components, exploit universal multipartite dependencies, certify full attainable rank structurally or over finite fields, and fall back to exact characteristic-zero elimination only for the genuinely difficult residual cores.

This framework recovers the standard two-way connected-component formula, exposes why pairwise connectivity is not sufficient from three fixed effects onward, and provides a theorem-by-theorem correspondence to an implementable HDFE rank engine. The remaining challenge is primarily computational—accelerating exact rank on very large, genuinely deficient irreducible cores—rather than identifying the mathematical target.

A Additional algebra for the three-way counterexample

For the matrix in (30), three independent column-null vectors are

$$\begin{aligned} z_1 &= (-1, -1, 1, 1, 0, 0, 0)', \\ z_2 &= (0, -1, -1, 0, 1, 1, 0)', \\ z_3 &= (-1, 0, 1, 0, 0, 0, 1)'. \end{aligned}$$

Therefore nullity is at least three and rank is at most $7 - 3 = 4$. The first, second, third, and fourth rows, for example, contain four independent directions, so rank equals four. Two of these null relations can be generated from the universal equality of block sums; the third is a collective dependency specific to the realized three-way incidence structure.

B Correctness of the native rational fallback

The dependency-free fallback performs sparse row elimination using integer cross-multiplication. Suppose a current row has pivot coefficient a in column c and the stored pivot row has coefficient $b \neq 0$. Replacing the current row by

$$\frac{b}{\gcd(|a|, |b|)}r - \frac{a}{\gcd(|a|, |b|)}p \tag{34}$$

zeros column c and is an exact rational elementary-row operation up to multiplication by a nonzero scalar. Dividing a resulting integer row by the gcd of all its entries and normalizing its sign also multiplies that row by a nonzero rational scalar. None of these operations changes row rank. Thus the number of pivots produced after exact elimination equals $\text{rank}_{\mathbb{Q}}$.

C Why a modular shortfall is not informative enough

It is possible for a matrix to have lower rank over $\mathbb{F}_p$ than over $\mathbb{Q}$ because a nonzero integer minor can become zero modulo p . A simple example is

$$B = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Over $\mathbb{Q}$, $\det(B) = -2$ and the rank is two. Modulo two the two rows coincide and the rank is one. This is why Algorithm 1 accepts finite-field computation only when it reaches an independent upper bound. The same logic applies to the GF(2) bitset pass in the implementation.

D Reproducibility notes

The audited prototype corresponding to this note exposes the following public-level operations:

```
categorical_rank(groups, backend="auto", return_info=False)
categorical_prefix_ranks(groups, backend="auto")
available_rank_backends()
```

The exact path uses no floating-point matrix-rank call. Optional backends are not required for correctness: they replace only the characteristic-zero fallback and can therefore affect speed without changing the target or the certification logic.

E Lean formalization of exact categorical rank

Scope and mathematical object. The formalized object is the actual finite categorical incidence matrix `incidenceMatrix`, with entries zero or one. Its rank is the dimension of the image of its linear map, not a matching-based rank of its sparsity pattern. The library is pinned to Lean 4.19.0 and Mathlib commit `c44e0c8ee63ca166450922a373c7409c5d26b00b`. A theorem is verified only for a source commit passing both the complete build and the per-theorem dependency audit.

Characteristic-zero equivalence. `matrix_exists_rank_minor` proves existence of a nonzero minor of order equal to the actual rank. Its row and column selections are injective; rank zero uses the empty minor of determinant one. The existence proof is not an executable search algorithm. Determinant transport gives `matrix_rank_map_field`, `rational_real_rank` and `integer_rational_real_rank`. In particular, `categorical_rational_real_rank` connects the exact rational rank of the categorical design to the real FE space used for projection.

Structural reductions. The duplicate-edge lemma is `categorical_rank_dedup`; its premise is coverage of every original tuple by a retained representative. Component additivity is `matrix_rank_components`, under actual row and column reindexings and vanishing cross-component entries. The rank recursion is `categorical_peeling_matrix_rank`; `categorical_leaf_rank` supplies its one-removal specialization. Each removed observation contributes one rank unit, even if it has several singleton levels. The proof obtains this fact from a bijection of fitted spaces, not from a rank assumption in the removal trace.

Block relations and the structural certificate. `multipartite_rank_drop` reconstructs the omitted columns from the retained complete reference block and the other columns in each partition. `multipartite_rank_upper` proves

$$\text{rank}(D) \leq \min\{E, V - (G - 1)\}.$$

`proper_connectivity_rank` proves equality with $V - (G - 1)$ when the sample is nonempty, every declared level is observed, and every two tuples are joined by a path changing at most one coordinate per step. This is stronger than ordinary incidence connectivity and is only a sufficient condition. The proof establishes a trivial kernel for the reduced matrix.

Finite-field lower bounds and exact acceptance. For an integer matrix A and a prime p , `modular_rank_le_charZero` and `integer_rational_real_rank` establish

$$\text{rank}_{\mathbb{F}_p}(A \bmod p) \leq \text{rank}_{\mathbb{Q}}(A) = \text{rank}_{\mathbb{R}}(A).$$

`modular_rank_exact` and `modular_minor_rank_exact` require that the modular lower bound reach an independently proved upper bound. `categorical_modular_rank_exact` supplies the categorical specialization. A modular shortfall is not a proof of characteristic-zero rank deficiency. `every_prime_has_rank_shortfall` gives a general integer-matrix counterexample for every prime; it is not presented as a full categorical design.

Mathematical composition. `categorical_structural_rank` connects tuple coverage, legal peeling and separated surviving blocks. `categorical_certified_rank` further uses mathematical rank evidence for every actual integer core submatrix:

$$\text{rank}_{\mathbb{R}}(D) = n_{\text{peeled}} + \sum_c r_c.$$

Each core count must be supported by a valid modular certificate or a completed legal rational row trace. The existing `exact_trace_rank` proves the algebraic correctness of such a trace. The composition theorem does not claim that a concrete backend generates the evidence. The algorithmic normal-return and resource-management aspects of Algorithm 1 are outside this appendix's scope.

Prefix allocation and weighting. `prefix_increment_total` and `prefix_redundancy_total` prove that nonnegative prefix increments telescope and that the redundancy allocation has the correct total. `matrix_prefix_redundancy_total` obtains each block-width bound from its actual matrix. `rank_column_permutation` proves total-rank invariance under reordering. `positive_weight_sqrt_rank` proves

$$\text{rank}(W^{1/2}D) = \text{rank}(D)$$

for strictly positive diagonal weights. These are structural dimension identities, not cluster-specific small-sample correction conventions.

Boundaries. Python, Numba, FLINT and SymPy implementations, modular elimination programs, gcd normalization code, execution time, memory budgets and failure-state refinement are not verification targets here. Source-code correspondence, floating-point accuracy, statistical assumptions and historical priority remain distinct from mathematical theorem verification. This appendix does not label every empirical illustration or every implementation statement in the manuscript as machine-checked.

References

- Abowd, J. M., Creecy, R. H., and Kramarz, F. (2002). Computing person and firm effects using linked longitudinal employer–employee data. Technical Paper TP-2002-06, U.S. Census Bureau, Center for Economic Studies.
- Correia, S. (2017). Linear models with high-dimensional fixed effects: An efficient and feasible estimator. Working paper. <https://scorreia.com/research/hdfe.pdf>.
- Correia, S. and Constantine, N. (2026). `reghdfe`: Stata module for linear and instrumental-variable/GMM regression absorbing multiple levels of fixed effects. Statistical Software Components S457874, Boston College Department of Economics; version 6.13.1 documentation and source repository. <https://github.com/sergiocorreia/reghdfe>.
- Correia, S., Guimarães, P., and Zylkin, T. (2020). Fast Poisson estimation with high-dimensional fixed effects. *The Stata Journal*, 20(1):95–115. doi:10.1177/1536867X20909691.
- FLINT Development Team (2026). FLINT documentation: integer matrices (`fmpz.mat`). Documentation accessed September 2026. https://flintlib.org/doc/fmpz_mat.html.
- Gaure, S. (2013). OLS with multiple high dimensional category variables. *Computational Statistics & Data Analysis*, 66:8–18. doi:10.1016/j.csda.2013.03.024.
- Guimarães, P. (2016). `GROUP3HDFE`: Stata module to compute number of restrictions in a linear regression model with three high-dimensional fixed effects. Statistical Software Components S458181, Boston College Department of Economics. <https://ideas.repec.org/c/boc/bocode/s458181.html>.
- Guimarães, P. and Portugal, P. (2010). A simple feasible procedure to fit models with high-dimensional fixed effects. *The Stata Journal*, 10(4):628–649. doi:10.1177/1536867X1101000406.
- Parui, S. (2025). On the incidence matrices of hypergraphs. *Linear and Multilinear Algebra*, 73(17). doi:10.1080/03081087.2025.2568155.
- SymPy Development Team (2026). DomainMatrix documentation: exact dense and sparse matrix arithmetic. Documentation accessed September 2026. <https://docs.sympy.org/latest/modules/polys/domainmatrix.html>.